UniPrefill: Universal Long-Context Prefill Acceleration via Block-wise Dynamic Sparsification

Qihang Fan[1,2,3,*], **Huaibo Huang**[1,2,†], **Zhiying Wu**[3],

Bingning Wang[3,‡], **Ran He**[1,2]

[1]MAIS&NLPR, CASIA [2]UCAS [3]WeChat, Tencent

https://github.com/qhfan/UniPrefill.git

Abstract

As large language models (LLMs) continue to advance rapidly, they are becoming increasingly capable while simultaneously demanding ever-longer context lengths. To improve the inference efficiency of long-context processing, several novel low-complexity hybrid architectures have recently been proposed, effectively alleviating the computational burden of long-context inference. However, existing research on long-context prefill acceleration remains predominantly focused on sparse attention mechanisms, which achieve their maximum speedup only on full-attention models. When transferred to emerging architectures — such as linear/full attention hybrids or sliding window/full attention hybrids — these prefill acceleration approaches suffer significant performance degradation. Furthermore, such methods are generally incompatible with continuous batching, making them difficult to integrate into modern inference engines such as vLLM. To this end, we propose **UniPrefill**, a prefill acceleration framework applicable to virtually any model architecture, which directly accelerates the model's computation at the token level. We further implement UniPrefill as a continuous batching operator and extend vLLM's scheduling strategy to natively support prefill-decode co-processing and tensor parallel for UniPrefill, enabling its seamless integration into vLLM. UniPrefill achieves up to **2.1x** speedup in Time-To-First-Token (TTFT), with the acceleration becoming increasingly pronounced as the number of concurrent requests grows.

1. Introduction

The rapid advancement of large language models (LLMs) has driven their deployment across an increasingly diverse range of real-world applications, from document understanding and code generation to multi-turn dialogue and retrieval-augmented generation [1, 24, 25, 32–34, 40]. Alongside this expansion in capability, the context lengths that LLMs are expected to

† Corresponding Author.
‡ Project Leader.
* Work done during internship at WeChat.

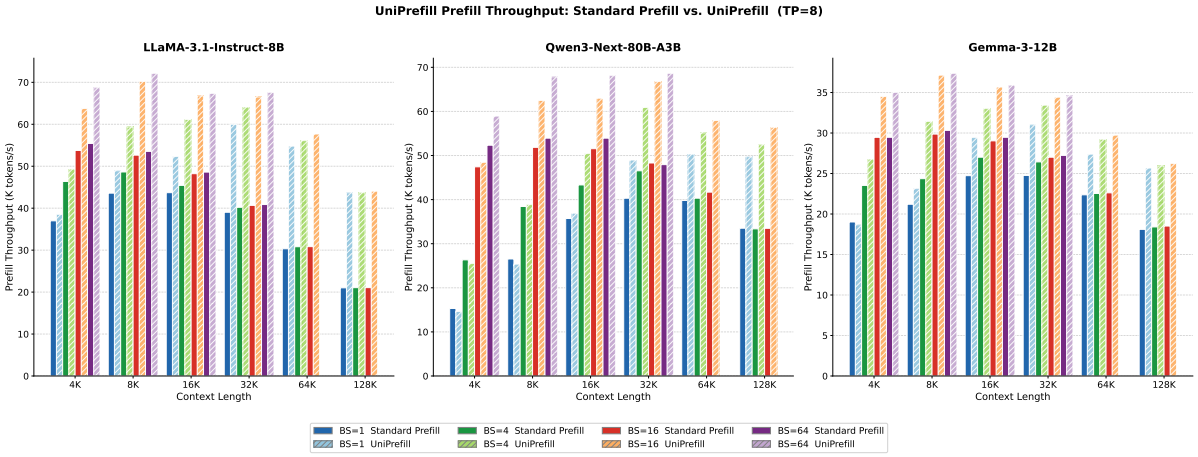

Figure 1 | Prefill throughput comparison between Standard Prefill and UniPrefill across three model architectures and varying batch sizes (tensor parallel size is set to 8). All experiments are conducted within vLLM, with UniPrefill deeply integrated into vLLM's continuous batching scheduler. We evaluate prefill throughput (K tokens/s) on LLaMA-3.1-Instruct-8B [4] (full attention), Qwen3-Next-80B-A3B [22] (linear/full attention hybrid), and Gemma-3-12B [14] (sliding window/full attention hybrid) across context lengths from 4K to 128K and batch sizes of 1, 4, 16, and 64. Solid bars denote Standard Prefill and hatched bars denote UniPrefill. UniPrefill consistently achieves higher throughput across all three architectures, with gains becoming more pronounced at longer context lengths and larger batch sizes.

process have grown dramatically — modern deployments routinely involve sequences of tens of thousands of tokens, and the demand for hundred-thousand-token or even million-token contexts is becoming commonplace. This trend places enormous pressure on inference efficiency, as the canonical Softmax Self-Attention [26] mechanism scales quadratically with sequence length, incurring prohibitive computational costs when processing long contexts.

To address the quadratic complexity bottleneck, a new generation of hybrid architectures has emerged that interleave computationally efficient layers with full attention layers. Two representative families have gained particular traction: linear/full attention hybrids, which replace a subset of attention layers with linear recurrent mechanisms [3, 5, 7, 10, 35] to reduce per-layer complexity from $O(N^2)$ to $O(N)$; and sliding window/full attention hybrids, which restrict most attention layers to a fixed local context window while retaining a small number of global full-attention layers for long-range dependencies [12, 14]. These hybrid designs substantially reduce the theoretical complexity of long-context inference and have been widely adopted in recently released production-grade models.

Despite the proliferation of hybrid architectures, the research community's efforts on prefill acceleration have remained heavily concentrated on sparse attention [8, 13, 21]. Representative works such as MInference [13] have demonstrated impressive prefill speedups, achieving up to 10× acceleration on long sequences under the full-attention-only setting. However, this focus on sparse attention comes with a fundamental limitation: the acceleration is tightly coupled to the full attention operation itself. In hybrid architectures where full attention constitutes only a fraction of all layers, the marginal benefit of accelerating solely those attention layers diminishes considerably. For instance, in a linear/full attention hybrid with a 3:1 ratio, at most one out of every four layers can be accelerated by existing sparse attention methods, leaving the dominant computational budget entirely untouched. This architectural mismatch renders existing prefill acceleration approaches far less effective on the new generation of hybrid models.

A second, equally critical limitation of existing prefill acceleration methods is their incompatibility with continuous batching, the scheduling paradigm that underpins modern high-throughput inference engines such as vLLM [15, 42]. Methods such as FlexPrefill [16] operate on individual requests in isolation and assume static batch composition, making them fundamentally difficult to integrate into a continuous batching scheduler where requests enter and exit the batch dynamically. As a result, these methods have largely remained research prototypes and have not been successfully embedded into production inference systems.

To overcome both limitations, we propose UniPrefill, a prefill acceleration framework that achieves architecture-agnostic speedups by exploiting a key insight: token importance can be estimated at full attention layers and propagated across all subsequent layers. Specifically, UniPrefill applies a lightweight block-wise scoring criterion at each full attention layer to identify and drop computationally redundant tokens. Once a token is dropped, it is excluded from all downstream computation in the remaining layers of the block. This cascading effect means that a single token-dropping decision at the attention layer translates into a proportional reduction in computation across the entire layer stack, not merely the attention sublayer. As a result, UniPrefill achieves substantial reductions in both attention FLOPs and GEMM FLOPs simultaneously, making it effective regardless of whether the model is a pure full-attention Transformer or hybrid architecture.

Beyond the algorithmic design, we address the systems integration challenge by implementing UniPrefill as a continuous batching operator [38] and extending vLLM [15]'s scheduler to natively support prefill-decode co-processing under UniPrefill's token-dropping regime. This tight integration allows UniPrefill to function as a transparent acceleration layer within production inference engines, without requiring changes to model weights or serving infrastructure.

We evaluate UniPrefill on RULER [11] with multiple model architectures. Results demonstrate that UniPrefill introduces no significant accuracy degradation while achieving up to 2.1× speedup in Time-To-First-Token (TTFT), as illustrated in Fig. 1. Notably, the speedup scales favorably with the number of concurrent requests (see Fig. 1), making UniPrefill particularly well-suited for high-concurrency production serving scenarios where prefill cost is the dominant bottleneck.

Our main contributions are summarized as follows:

- We propose **UniPrefill**, a token-level prefill acceleration framework that drops tokens at full attention layers and propagates sparsity across all subsequent layers, reducing both attention and GEMM FLOPs simultaneously, which enables consistent speedups across heterogeneous hybrid architectures.
- We implement UniPrefill as a continuous batching operator and integrate it into vLLM [15] via extended scheduling strategies that support prefill-decode co-processing and tensor parallel, enabling seamless production-ready deployment.
- Extensive experiments on the long context benchmark RULER demonstrate that UniPrefill achieves up to 2.1× TTFT speedup with negligible accuracy loss, with acceleration gains scaling with request concurrency.

2. Related Works

Hybrid LLM Architectures. To overcome the quadratic complexity of Softmax attention, a rich body of work has proposed efficient sequence modeling alternatives, including state space models, linear attention variants, and recurrent architectures [3, 5, 6, 10, 18, 23, 35–37, 41]. To

balance efficiency and expressiveness, hybrid architectures have emerged that interleave full attention with these efficient alternatives [12, 14, 17, 22, 28], and have been widely adopted in recently released production models. However, existing prefill acceleration methods remain largely tailored to full-attention-only architectures, limiting their effectiveness on this new generation of models.

Sparse Attention for Prefill Acceleration. Exploiting the inherent sparsity in attention score matrices is a well-established strategy for accelerating the prefill stage. A body of work identifies static or dynamic sparse patterns — such as vertical, slash, and block-sparse structures — and skips the corresponding attention computations [2, 13, 16, 21, 30, 39]. These methods have demonstrated substantial speedups on full attention models [13, 16, 27, 31]. However, they share two fundamental limitations: their acceleration is tightly coupled to the attention operation itself, leaving FFN and GEMM computations entirely unaccelerated, and they are generally incompatible with continuous batching [38], making integration into production inference engines such as vLLM [15] non-trivial. UniPrefill addresses both limitations by operating at the token level and propagating sparsity across all layers.

3. Method

In this section, we present UniPrefill, an architecture-agnostic prefill acceleration framework. The overall pipeline is illustrated in Fig. 2.

3.1. Preliminaries

Consider an input sequence $\mathbf{x} = [x_1, \ldots, x_N]$ processed by a hybrid LLM consisting of B blocks. Each block b contains a full attention layer followed by M_b sublayers (linear attention, sliding window attention, FFN, etc.). Let $\mathbf{H}^{(b,0)} \in \mathbb{R}^{N\times d}$ denote the block input. The goal of prefill is to compute the final hidden state $\mathbf{h}_N^{(L)}$ for next-token prediction:

$$P(x_{N+1} \mid x_{1:N}) = \mathrm{LMHead}\left(\mathbf{h}_N^{(L)}\right) \tag{1}$$

Standard prefill incurs $O(N^2 d_k)$ per full attention layer and $O(Nd^2)$ per GEMM sublayer, totaling $O(N^2 d_k + M_b N d^2)$ per block.

3.2. Token Importance Estimation

Since next-token prediction depends solely on $\mathbf{h}_N^{(L)}$, the contribution of token i to the final hidden state at block b is:

$$\mathbf{h}_N^{(b,1)} = \sum_{i=1}^{N} \mathbf{A}_{N,i}^{(b)} \cdot \mathbf{v}_i^{(b)} + \mathbf{h}_N^{(b,0)}, \tag{2}$$

where $\mathbf{A}_{N,i}^{(b)} = \mathrm{softmax}_i\left(\mathbf{q}_N^{(b)} \mathbf{K}^{(b)\top} / \sqrt{d_k}\right)$ is the full-sequence attention weight. A token i is negligible to next-token prediction when $\mathbf{A}_{N,i}^{(b)} \approx 0$.

To reduce estimation variance, we aggregate over the last n query positions instead of a single position:

$$s_i^{(b)} = \frac{1}{n} \sum_{j=N-n+1}^{N} \mathbf{A}_{j,i}^{(b)}, \tag{3}$$

requiring an $n \times N$ attention computation at cost $O(nNd_k)$, negligible for $n \ll N$.

In practice, importance estimation and token selection operate at block granularity. We partition the input sequence into non-overlapping blocks of size G: $\mathcal{B}_g = \{(g-1)G+1, \ldots, \min(gG, N)\}$, $g = 1, \ldots, \lceil N/G \rceil$. For efficiency, the partial GEMM $\mathbf{S} = \mathbf{Q}_{[N-n:N]}\mathbf{K}^\top \in \mathbb{R}^{n \times N}$ is computed first; an online softmax is then applied across the full sequence dimension to obtain properly normalised attention weights, after which scores are reduced within each block:

$$\bar{s}_g^{(b)} = \frac{1}{G} \sum_{i \in \mathcal{B}_g} \frac{1}{n} \sum_{j=N-n+1}^{N} \mathbf{A}_{j,i}^{(b)}, \tag{4}$$

where the softmax normalisation is performed over the complete key sequence before the block reduction, ensuring $\bar{s}_g^{(b)}$ reflects the true attention mass captured by block g. This reduces the number of selection decisions from N to $\lceil N/G \rceil$ while preserving the accuracy of importance estimation.

Relationship to SnapKV. Our importance estimation shares a surface-level similarity with SnapKV [19], which also uses an observation window to identify important tokens. However, the two methods differ fundamentally in objective and scope. SnapKV completes a full $N \times N$ prefill across all layers before applying its selection to compress the KV cache for decode—the prefill FLOPs are entirely unaffected. UniPrefill applies selection during prefill, propagating the drop decision forward through all subsequent layers. Formally, whereas SnapKV saves at most $O(N \cdot d_{kv})$ in decode-time memory per layer, UniPrefill saves $(1 - \rho^{(b)}) \cdot M_b \cdot O(Nd^2)$ in prefill-time FLOPs per block, where $\rho^{(b)} = |\mathcal{S}^{(b)}|/N$ is the token retention ratio—a quantity that grows linearly with M_b and is entirely absent in SnapKV.

3.3. Top-p Token Selection

Let π be the permutation sorting block-level scores $\{\bar{s}_g^{(b)}\}$ in descending order. We retain the minimal set of blocks:

$$\mathcal{S}^{(b)} = \{\pi(1), \ldots, \pi(k^*)\}, \qquad k^* = \min k \ \text{ s.t. } \frac{\sum_{j=1}^{k} \bar{s}_{\pi(j)}^{(b)}}{\sum_g \bar{s}_g^{(b)}} \geqslant p \tag{5}$$

The dropped set is $\bar{\mathcal{S}}^{(b)} = [N] \setminus \mathcal{S}^{(b)}$. Two structural elements are always retained regardless of their scores: the first A tokens (attention sinks [29]) and the last n tokens (the query window itself), ensuring causal consistency and numerical stability.

Error bound. The perturbation to any retained position j due to dropping $\bar{\mathcal{S}}^{(b)}$ satisfies:

$$\left\| \Delta \mathbf{h}_j^{(b,1)} \right\| \leqslant \left(\sum_{i \in \bar{\mathcal{S}}^{(b)}} \mathbf{A}_{j,i}^{(b)} \right) \cdot V_{\max}^{(b)} \leqslant (1-p) \cdot V_{\max}^{(b)} \tag{6}$$

where $V_{\max}^{(b)} = \max_i \|\mathbf{v}_i^{(b)}\|$. Setting $p = 0.99$ guarantees that at most 1% of the total attention mass is discarded, providing a direct information-theoretic bound on the approximation error at the attention layer.

Top-p vs. top-k. A fixed top-k is insensitive to the actual distribution of attention: when attention is highly concentrated, top-k retains many unnecessary tokens; when diffuse, it may drop tokens with non-trivial contributions. Top-p adapts automatically—the retained set is small when attention is concentrated and large when it is diffuse—providing a uniform bound on approximation error regardless of sequence length or content, which top-k cannot guarantee.

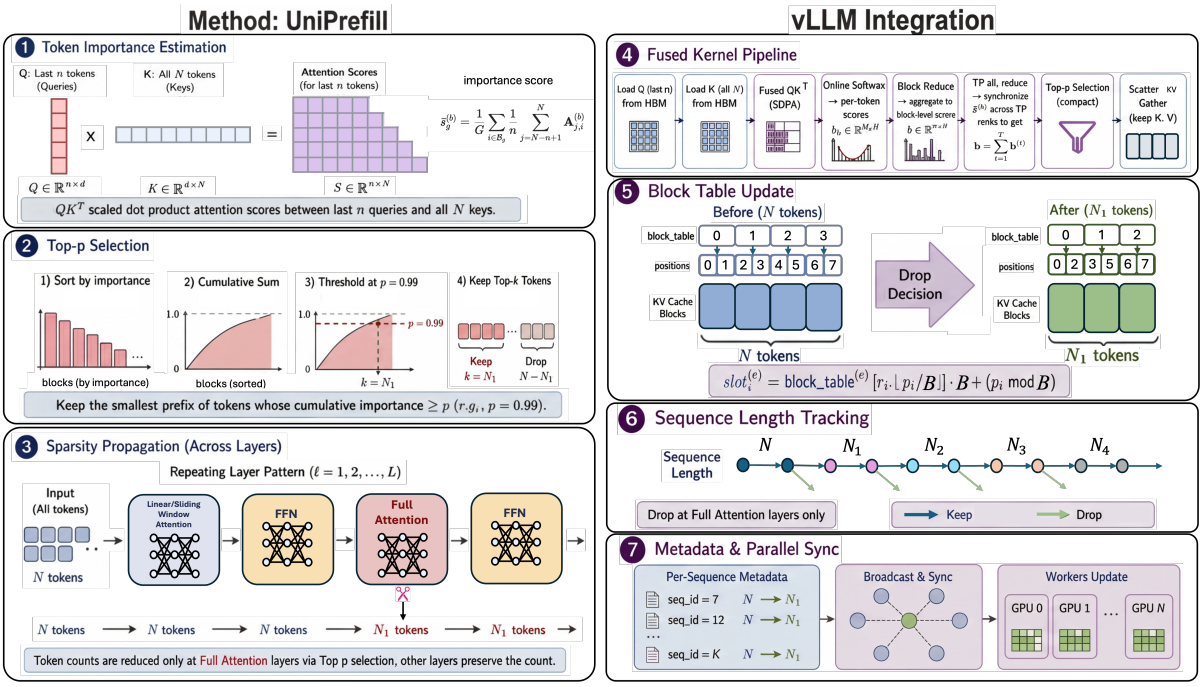

Figure 2 | **Overview of UniPrefill. Left:** UniPrefill estimates token importance via block-level attention scores from the last n queries **(1)**, retains the smallest set of token blocks whose cumulative importance reaches p **(2)**, and propagates the resulting sparsity across all subsequent sub-layers within each repeating layer pattern **(3)**. **Right:** UniPrefill is deeply integrated into vLLM via a fused kernel pipeline **(4)**, with KV cache block tables **(5)**, per-layer sequence length tracking **(6)**, and tensor-parallel metadata synchronisation **(7)** updated accordingly.

3.4. Sparsity Propagation Across All Layers

After token selection at the full attention layer of block b, dropped tokens $\bar{\mathcal{S}}^{(b)}$ are excluded from all subsequent sublayers within and beyond the block—every full attention, linear attention, sliding window attention, and FFN layer processes only the retained set $\mathcal{S}^{(b)}$:

$$\mathbf{H}_{\mathcal{S}}^{(b,m+1)} = f_m\left(\mathbf{H}_{\mathcal{S}}^{(b,m)}\right), \qquad m = 1, \ldots, M_b \tag{7}$$

At block $b+1$, the full sequence is reconstituted by carrying dropped token states forward without update:

$$\mathbf{H}_i^{(b+1,0)} = \begin{cases} \mathbf{H}_i^{(b,M_b+1)} & i \in \mathcal{S}^{(b)} \\ \mathbf{H}_i^{(b,0)} & i \in \bar{\mathcal{S}}^{(b)} \end{cases} \tag{8}$$

and importance scores are recomputed fresh at each block's full attention layer. This means a single drop decision at layer ℓ immediately reduces the token count for all layers $\ell' > \ell$, including subsequent full attention layers, linear attention layers, sliding window layers, and all FFN projections.

FLOPs analysis. Let $\mathcal{L}_{\text{drop}} = \{\ell_1, \ell_2, \ldots\}$ denote the set of layers at which dropping is applied, and let ρ_k denote the retention ratio after the k-th drop. The total FLOPs saved across all L layers is:

$$\Delta\text{FLOPs} = \sum_k (1 - \rho_k) \cdot \sum_{\ell > \ell_k} \text{FLOPs}_\ell(N) \tag{9}$$

For a model with L total layers each of cost $O(Nd^2)$, a single drop at layer ℓ_1 with retention ratio ρ saves:

$$\Delta\text{FLOPs}^{(\ell_1)} = (1 - \rho) \cdot (L - \ell_1) \cdot O(Nd^2) \tag{10}$$

This saving scales linearly with $(L - \ell_1)$, the number of layers remaining after the drop point. Sparse attention methods operating only within the attention sublayer save at most $(1 - \rho) \cdot O(N^2 d_k)$ at that layer alone, leaving all subsequent GEMM costs intact. The ratio of savings is:

$$\frac{\Delta\text{FLOPs}_{\text{UniPrefill}}}{\Delta\text{FLOPs}_{\text{SparseAttn}}} = \frac{(L - \ell_1) \cdot Nd^2}{N^2 d_k} \xrightarrow{N \to \infty} \infty \tag{11}$$

In the long-context regime where $N \gg d$, UniPrefill's GEMM savings dominate, making it particularly effective precisely at the sequence lengths where prefill acceleration matters most.

Error propagation. Assuming each sublayer f_m is L_m-Lipschitz, the accumulated error at block end satisfies:

$$\left\| \Delta \mathbf{h}_j^{(b,M_b+1)} \right\| \leqslant (1 - p) \cdot V_{\max}^{(b)} \cdot \prod_{m=1}^{M_b} L_m \tag{12}$$

Layer normalization and residual connections constrain $\prod_m L_m$ in practice, preventing unbounded error amplification across layers.

3.5. Fused Kernel and vLLM Integration

Kernel design. We implement the importance estimation and top-p selection pipeline as a sequence of four fused kernels operating directly on the variable-length packed token representation indexed by `cu_seqlens`, without materializing per-request tensors or padding. The

pipeline proceeds as follows:

$$\mathbf{S} = \mathbf{Q}_{[N-n:N]}\mathbf{K}^\top \in \mathbb{R}^{n\times N} \xrightarrow{\text{online softmax}} \mathbf{o} \in \mathbb{R}^{N} \xrightarrow{\text{block reduce}} \mathbf{b} \in \mathbb{R}^{\lceil N/G \rceil} \xrightarrow{\text{top-}p} \mathcal{M} \in \{0,1\}^{N} \tag{13}$$

The partial GEMM kernel computes $\mathbf{S}$ with tiled Q-K blocking and inline causal masking. The softmax kernel aggregates softmax($\mathbf{S}$) over the n query rows via a numerically stable two-pass online algorithm, yielding per-token importance scores $\mathbf{o}$. The block-reduce kernel contracts $\mathbf{o}$ across both the head and spatial dimensions within each block of size G, producing the block-level score vector $\mathbf{b}$.

The top-p kernel performs sort-and-threshold entirely on-GPU without CPU round-trips. We encode each (score, index) pair into a single `int64` word via a monotone IEEE-754 bitcast mapping:

$$\varphi(x) = \begin{cases} \text{bits}(x) \oplus \texttt{0x80000000} & x \geqslant 0 \\ \text{bits}(x) \oplus \texttt{0xFFFFFFFF} & x < 0 \end{cases} \quad \Rightarrow \quad \texttt{packed} = \big(\varphi(b_g) \ll 32\big) \,\big|\, g \tag{14}$$

Sorting packed words descending, computing a cumulative sum of scores, and thresholding at p yields the keep mask $\mathcal{M}$, which is scattered back to original positions. A final expansion kernel lifts $\mathcal{M}$ from block to token granularity, unconditionally setting $\mathcal{M}_i = 1$ for attention-sink tokens $i < A$ and query-window tokens $i \geqslant N - n$.

Tensor parallelism. Under tensor parallelism of degree T, each rank observes only $1/T$ of the attention heads, yielding a partial block score $\mathbf{b}^{(t)}$. We synchronize via:

$$\mathbf{b} = \sum_{t=1}^{T} \mathbf{b}^{(t)} \tag{15}$$

before the top-p kernel, ensuring a consistent drop decision across all TP ranks.

vLLM scheduler integration. Integrating token dropping into vLLM's continuous batching scheduler [15, 38] requires maintaining correctness across three coupled state structures: layer-wise attention metadata, KV cache slot mappings, and per-request KV length tracking across decode steps.

Upon a drop event at layer ℓ, we propagate updated metadata to all downstream layers $\ell' > \ell$ by patching `query_start_loc`, `seq_lens`, and `num_actual_tokens` to reflect the compacted token stream $|\mathcal{S}^{(\ell)}|$. Physical KV cache slot mappings for each layer ℓ' are recomputed as:

$$\texttt{slot}_i^{(\ell')} = \texttt{block\_table}^{(\ell')}[r_i,\, \lfloor p_i/B \rfloor] \cdot B + (p_i \bmod B) \tag{16}$$

where p_i is the logical position of the i-th retained token, B is the KV block size, and $\texttt{block\_table}^{(\ell')}$ is the physical block table of layer ℓ'—which may differ between global and sliding-window attention layers [14].

During decode, each layer ℓ' must attend over only the tokens that were physically written to its KV cache during prefill. We maintain a per-request drop history $\{(\ell_k, s_k^r)\}$ recording the retained sequence length s_k^r after each drop event at layer ℓ_k. The effective KV length visible to layer ℓ' during decode is then:

$$\text{seqused}_r^{(\ell')} = s_r^{(\ell^-)} + \Delta_r, \qquad \Delta_r = \text{kv\_len}_r - \text{orig\_len}_r \tag{17}$$

Method	4K	8K	16K	32K	64K	128K	Avg	4K	8K	16K	32K	64K	128K
Llama-3.1-8B-Instruct (Full Attention)													
Baseline	97.36	95.98	94.62	91.02	86.29	76.89	90.36	1.00×	1.00×	1.00×	1.00×	1.00×	1.00×
LazyLLM [9]	89.16	81.12	70.32	64.39	56.28	49.71	68.50	1.09×	1.19×	1.28×	1.74×	2.21×	2.51×
SlimInfer [20]	90.23	82.04	71.39	67.12	57.10	45.36	68.87	1.07×	1.16×	1.25×	1.66×	1.98×	2.07×
MInference [13]	96.71	95.78	95.51	90.76	87.12	78.21	90.68	0.82×	0.86×	0.98×	1.03×	1.15×	1.34×
FlexPrefill [16]	96.34	95.12	94.83	88.96	84.31	78.13	89.62	0.83×	0.89×	1.02×	1.08×	1.24×	1.46×
XAttention [31]	95.98	95.23	94.68	88.06	83.92	78.16	89.34	0.92×	0.96×	1.03×	1.08×	1.21×	1.38×
ProxyAttn [27]	96.78	95.46	95.49	89.28	85.31	78.49	90.14	0.82×	0.89×	1.03×	1.11×	1.36×	1.79×
UniPrefill	96.53	95.83	95.41	89.77	85.28	79.87	**90.45**	1.21×	1.34×	1.37×	1.62×	2.01×	2.26×
Qwen3-Next-80B-A3B (Linear/Full Attention Hybrid)													
Baseline	96.83	95.67	95.07	94.38	94.51	92.09	94.76	1.00×	1.00×	1.00×	1.00×	1.00×	1.00×
LazyLLM [9]	89.13	82.37	70.69	64.37	58.12	55.17	69.98	1.11×	1.18×	1.29×	1.40×	1.55×	1.74×
SlimInfer [20]	88.11	80.36	67.13	63.22	57.13	55.36	68.55	1.14×	1.22×	1.27×	1.34×	1.42×	1.56×
MInference [13]	96.62	94.38	94.49	94.27	94.28	91.81	94.31	0.96×	0.98×	1.00×	1.00×	1.02×	1.05×
FlexPrefill [16]	96.36	95.03	94.17	93.91	92.89	91.44	93.97	0.96×	0.98×	1.00×	1.01×	1.04×	1.08×
XAttention [31]	96.03	94.81	94.03	93.01	93.06	90.23	93.53	0.97×	0.99×	1.00×	1.00×	1.02×	1.05×
ProxyAttn [27]	96.31	94.13	94.23	93.47	93.51	91.62	93.88	0.96×	0.98×	1.00×	1.02×	1.05×	1.11×
UniPrefill	96.67	94.49	94.29	93.63	93.13	91.41	**93.94**	1.08×	1.21×	1.24×	1.39×	1.42×	1.68×
Gemma-3-12B (Sliding Window/Full Attention Hybrid)													
Baseline	94.01	89.12	85.98	80.76	68.89	61.22	79.99	1.00×	1.00×	1.00×	1.00×	1.00×	1.00×
LazyLLM [9]	86.11	80.34	75.23	68.42	54.12	43.38	67.93	1.23×	1.32×	1.37×	1.42×	1.53×	1.64×
SlimInfer [20]	88.23	83.14	79.12	69.33	53.02	40.11	68.83	1.15×	1.24×	1.32×	1.39×	1.45×	1.52×
MInference [13]	93.56	89.51	86.01	80.04	67.09	59.31	79.25	0.98×	0.99×	1.00×	1.00×	1.01×	1.03×
FlexPrefill [16]	93.63	89.16	85.49	79.23	65.69	58.63	78.64	0.98×	0.99×	1.00×	1.01×	1.02×	1.04×
XAttention [31]	93.06	89.24	85.67	79.14	66.18	56.24	78.26	0.99×	1.00×	1.01×	1.01×	1.02×	1.02×
ProxyAttn [27]	93.67	89.52	85.31	78.97	65.31	59.93	78.79	0.98×	0.99×	1.01×	1.01×	1.03×	1.06×
UniPrefill	93.18	89.76	86.47	79.08	66.32	58.38	**78.87**	1.15×	1.21×	1.22×	1.26×	1.31×	1.49×

Table 1 | Performance vs. efficiency across different models and methods. Evaluation scores (left) and TTFT speedup relative to baselines (right) are reported. For a fair comparison, all models are evaluated using HuggingFace Transformers with batch size equals to 1.

where $\ell^- = \max\{\ell_k \in \mathcal{L}_{\text{drop}} : \ell_k < \ell'\}$ is the last drop layer preceding ℓ', and Δ_r counts autoregressive tokens appended since prefill. This per-layer `seqused` correction is injected into the forward context before each decode step, ensuring every attention layer observes a KV sequence length precisely consistent with its written cache entries—without any modification to model weights or the PagedAttention memory allocator.

4. Experiments

We evaluate UniPrefill across two dimensions: accuracy and efficiency. For accuracy, we compare UniPrefill against existing prefill acceleration methods on the RULER [11] long-context benchmark across multiple model architectures. For efficiency, we measure prefill throughput under varying context lengths and batch sizes within our vLLM deployment. Finally, we conduct ablation studies to analyze the contribution of each design choice in UniPrefill. Implementation and deployment details can be found in appendix.

BSZ	4K	8K	16K	32K	64K	128K	4K	8K	16K	32K	64K	128K
Llama-3.1-8B-Instruct (Full Attention)												
1	36984	43586	43697	39027	30324	21013	38522 (+4%)	48932 (+12%)	52314 (+20%)	59984 (+54%)	54786 (+81%)	43672 (+107%)
4	46372	48632	45436	40210	30812	21054	49336 (+6%)	59372 (+22%)	61148 (+35%)	64113 (+59%)	56108 (+82%)	43698 (+108%)
16	53764	52643	48221	40671	30834	21062	63762 (+19%)	70213 (+33%)	66762 (+38%)	66512 (+64%)	57678 (+87%)	44042 (+109%)
64	55431	53541	48603	40869	—	—	68721 (+24%)	72139 (+35%)	67361 (+39%)	67618 (+65%)	—	—
Qwen3-Next-80B-A3B (Linear/Full Attention Hybrid)												
1	15314	26534	35712	40312	39807	33512	14621 (−5%)	25364 (−4%)	36912 (+3%)	48972 (+21%)	50324 (+26%)	49732 (+48%)
4	26334	38442	43326	46534	40322	33364	25498 (−3%)	38872 (+1%)	50442 (+16%)	60894 (+31%)	55136 (+37%)	52442 (+57%)
16	47432	51853	51544	48303	41693	33489	48446 (+2%)	62468 (+20%)	62983 (+22%)	66798 (+38%)	57938 (+39%)	56398 (+68%)
64	52334	53936	53936	47932	—	—	58936 (+13%)	67848 (+26%)	68123 (+26%)	68631 (+43%)	—	—
Gemma-3-12B (Sliding Window/Full Attention Hybrid)												
1	19013	21203	24733	24763	22367	18103	18673 (−2%)	23132 (+9%)	29436 (+19%)	31023 (+25%)	27384 (+22%)	25673 (+42%)
4	23531	24361	27013	26432	22536	18403	26783 (+14%)	31432 (+29%)	33014 (+22%)	33432 (+26%)	29232 (+30%)	25932 (+41%)
16	29468	29867	29031	27012	22613	18513	34512 (+17%)	37136 (+24%)	35674 (+23%)	34419 (+27%)	29732 (+31%)	26231 (+42%)
64	29471	30324	29471	27236	—	—	35016 (+19%)	37365 (+23%)	35912 (+22%)	34578 (+27%)	—	—

Table 2 | Prefill throughput (tokens/s) of Standard Prefill and UniPrefill measured within vLLM (TP= 8) across three model architectures, six context lengths (4K–128K), and four batch sizes (BSZ). The left half of each group reports Standard Prefill throughput; the right half reports UniPrefill throughput.

4.1. Experimental Setup

We select three model architectures to validate the effectiveness of UniPrefill: LLaMA-3.1-8B-Instruct [4], which consists entirely of full-attention layers; Qwen3-Next-80B-A3B [22], a linear/full-attention hybrid with a 3:1 ratio; and Gemma-3-12B [14], a sliding-window/full-attention hybrid with a 5:1 ratio. We set the top-p threshold to 0.99, 0.99, and 0.98 for the three models, respectively. The minimum dropping granularity is set to a block size of $G = 64$ tokens, and importance scores are estimated using the last $n = 128$ query tokens. To preserve attention sinks [29], the first 128 tokens are always retained.

4.2. Results on RULER

RULER [11] is a comprehensive long-context benchmark that evaluates LLMs across diverse task categories including retrieval, multi-hop tracing, aggregation, and question answering, with configurable context lengths up to 128K tokens. Unlike prior benchmarks that rely on simple needle-in-a-haystack tests, RULER provides a more rigorous and systematic assessment of true long-context understanding, making it a widely adopted standard for evaluating long-context LLM performance.

block size	4K	8K	16K	32K	64K	128K	Avg	4K	8K	16K	32K	64K	128K
Llama-3.1-8B-Instruct													
64	96.53	95.83	95.41	89.77	85.28	79.87	90.45	+19%	+33%	+38%	+64%	+87%	+109%
128	94.32	93.62	93.07	88.12	83.38	78.90	88.57	+26%	+38%	+45%	+62%	+81%	+96%
32	93.42	94.63	95.46	90.23	85.67	79.88	89.88	+19%	+32%	+36%	+72%	+96%	+121%
Qwen3-Next-80B-A3B													
64	96.67	94.49	94.29	93.63	93.13	91.41	93.94	+2%	+20%	+22%	+38%	+39%	+68%
128	96.52	94.69	94.13	93.41	92.67	92.06	93.91	+5%	+22%	+23%	+34%	+37%	+56%
32	92.17	94.88	94.72	93.89	93.66	92.68	93.67	+0%	+19%	+22%	+44%	+51%	+78%

Table 3 | Ablation study of block size G. The left panel reports RULER scores under different values of G, and the right panel reports the corresponding TTFT speedup of UniPrefill relative to the Baseline.

Tab. 1 presents RULER scores and TTFT speedups across three model architectures. UniPrefill achieves the best accuracy-efficiency tradeoff among all acceleration methods. LazyLLM and SlimInfer suffer notable accuracy degradation across all three architectures, while sparse attention methods preserve accuracy but yield diminishing speedups on hybrid architectures, with gains often below 1.1× at 128K. UniPrefill strikes the optimal balance: it retains accuracy close to the Baseline while delivering up to 2.26×, 1.68×, and 1.49× TTFT speedup at 128K context length on LLaMA-3.1-8B, Qwen3-Next-80B-A3B, and Gemma-3-12B, respectively, demonstrating consistent effectiveness across full-attention and hybrid architectures.

4.3. vLLM Intergration

Tab. 2 reports prefill throughput within vLLM across three architectures. UniPrefill consistently improves throughput as context length and batch size increase, achieving up to +109%, +68%, and +42% gains on LLaMA-3.1-8B, Qwen3-Next-80B-A3B, and Gemma-3-12B, respectively. The speedup scales favorably with both context length and batch size, demonstrating that UniPrefill is particularly effective in the high-concurrency, long-context regime that dominates production serving workloads.

4.4. Ablation Study

Block Size. Tab. 3 presents the ablation results for block size $G \in \{32, 64, 128\}$. At short context lengths, $G = 128$ yields the highest speedup due to lower selection overhead per drop decision. As context length grows, $G = 32$ surpasses $G = 128$ in speedup, since finer granularity allows more tokens to be dropped, achieving up to +121% and +78% throughput gain on LLaMA-3.1-8B and Qwen3-Next-80B-A3B at 128K, respectively. We adopt $G = 64$ as the default, which balances selection overhead and drop rate across all context lengths.

Last n. Tab. 4 reports RULER scores under different values of last $n \in \{32, 128, 512\}$. $n = 32$ leads to a noticeable accuracy drop, as too few query tokens introduce high variance in importance estimation. $n = 512$ recovers accuracy but incurs higher computational overhead. $n = 128$ achieves the best balance and is adopted as the default.

last n	4K	8K	16K	32K	64K	128K	Avg
Llama-3.1-8B-Instruct							
128	96.53	95.83	95.41	89.77	85.28	79.87	90.45
32	95.32	94.13	93.18	86.22	82.63	75.13	87.77
512	96.72	96.04	95.63	90.23	84.96	79.38	90.49

Table 4 | Ablation study of last n. RULER scores under different values of the n on LLaMA-3.1-8B-Instruct. $n = 128$ is adopted as the default.

5. Conclusion

We present UniPrefill, an architecture-agnostic framework for long-context LLM prefill acceleration. By estimating token importance via block-wise top-p selection at full-attention layers and propagating the sparsity mask across all subsequent sub-layers, UniPrefill simultaneously reduces attention and GEMM FLOPs, making it effective across full-attention and hybrid architectures alike. We implement UniPrefill as a fused kernel pipeline and integrate it into vLLM's continuous-batching scheduler without any model weight changes. Experiments on the RULER benchmark show that UniPrefill achieves the best accuracy-efficiency tradeoff among all compared methods, delivering up to 2.1× TTFT speedup with negligible accuracy loss, with gains scaling favorably with context length and batch size. We hope UniPrefill provides a practical and general solution for efficient long-context LLM serving.

References

[1] J. Bai, S. Bai, Y. Chu, Z. Cui, K. Dang, X. Deng, Y. Fan, W. Ge, Y. Han, F. Huang, B. Hui, L. Ji, M. Li, J. Lin, R. Lin, D. Liu, G. Liu, C. Lu, K. Lu, J. Ma, R. Men, X. Ren, X. Ren, C. Tan, S. Tan, J. Tu, P. Wang, S. Wang, W. Wang, S. Wu, B. Xu, J. Xu, A. Yang, H. Yang, J. Yang, S. Yang, Y. Yao, B. Yu, H. Yuan, Z. Yuan, J. Zhang, X. Zhang, Y. Zhang, Z. Zhang, C. Zhou, J. Zhou, X. Zhou, and T. Zhu. Qwen technical report. arXiv preprint arXiv:2309.16609, 2023. URL https://arxiv.org/abs/2309.16609.

[2] G. Chen. VSPrefill: Vertical-slash sparse attention with lightweight indexing for long-context prefilling. arXiv preprint arXiv:2603.04460, 2026.

[3] T. Dao and A. Gu. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. In International Conference on Machine Learning, 2024.

[4] A. Dubey, A. Jauhri, A. Pandey, et al. The llama 3 herd of models. arXiv preprint arXiv:2407.21783, 2024.

[5] Q. Fan, H. Huang, Y. Ai, and R. He. Rectifying magnitude neglect in linear attention. In IEEE International Conference on Computer Vision, 2025.

[6] Q. Fan, H. Huang, M. Chen, and R. He. Semantic equitable clustering: A simple and effective strategy for clustering vision tokens. In IEEE International Conference on Computer Vision, 2025.

[7] Q. Fan, H. Huang, and R. He. Breaking the low-rank dilemma of linear attention. In IEEE Conference on Computer Vision and Pattern Recognition, 2025.

[8] Q. Fan, H. Huang, Z. Wu, J. Wang, B. Wang, and R. He. Flashprefill: Instantaneous pattern discovery and thresholding for ultra-fast long-context prefilling. arXiv preprint arXiv:2603.06199, 2026. URL https://arxiv.org/abs/2603.06199.

[9] Q. Fu, M. Cho, T. Merth, S. Mehta, M. Rastegari, and M. Najibi. Lazyllm: Dynamic token pruning for efficient long context llm inference. arXiv preprint arXiv:2407.14057, 2024. URL https://arxiv.org/abs/2407.14057.

[10] A. Gu and T. Dao. Mamba: Linear-time sequence modeling with selective state spaces. arXiv preprint arXiv:2312.00752, 2023.

[11] C.-P. Hsieh, S. Sun, S. Kriman, S. Acharya, D. Rekesh, F. Jia, Y. Zhang, and B. Ginsburg. Ruler: What's the real context size of your long-context language models? arXiv preprint arXiv:2404.06654, 2024. URL https://arxiv.org/abs/2404.06654.

[12] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L. R. Lavaud, M.-A. Lachaux, P. Stock, T. L. Scao, T. Lavril, T. Wang, T. Lacroix, and W. E. Sayed. Mistral 7b. arXiv preprint arXiv:2310.06825, 2023. URL https://arxiv.org/abs/2310.06825.

[13] H. Jiang, Y. Li, C. Zhang, Q. Wu, X. Luo, S. Ahn, Z. Han, A. H. Abdi, D. Li, C.-Y. Lin, Y. Yang, and L. Qiu. MInference 1.0: Accelerating pre-filling for long-context LLMs via dynamic sparse attention. In Advances in Neural Information Processing Systems, 2024.

[14] A. Kamath, J. Ferret, S. Pathak, et al. Gemma 3 technical report. arXiv preprint arXiv:2503.19786, 2025. URL https://arxiv.org/abs/2503.19786.

[15] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with pagedattention. In Proceedings of the ACM SIGOPS Symposium on Operating Systems Principles, 2023.

[16] X. Lai, J. Lu, Y. Luo, Y. Ma, and X. Zhou. Flexprefill: A context-aware sparse attention mechanism for efficient long-sequence inference. In International Conference on Learning Representations, 2025.

[17] B. Lenz, O. Lieber, A. Arazi, A. Bergman, A. Manevich, B. Peleg, B. Aviram, et al. Jamba: Hybrid Transformer-Mamba language models. In ICLR, 2025.

[18] A. Li, B. Gong, B. Yang, B. Shan, C. Liu, C. Zhu, C. Zhang, C. Guo, D. Chen, D. Li, E. Jiao, G. Li, G. Zhang, H. Sun, H. Dong, J. Zhu, J. Zhuang, J. Song, J. Zhu, J. Han, J. Li, J. Xie, J. Xu, J. Yan, K. Zhang, K. Xiao, K. Kang, L. Han, L. Wang, L. Yu, L. Feng, L. Zheng, L. Chai, L. Xing, M. Ju, M. Chi, M. Zhang, P. Huang, P. Niu, P. Li, P. Zhao, Q. Yang, Q. Xu, Q. Wang, Q. Wang, Q. Li, R. Leng, S. Shi, S. Yu, S. Li, S. Zhu, T. Huang, T. Liang, W. Sun, W. Sun, W. Cheng, W. Li, X. Song, X. Su, X. Han, X. Zhang, X. Hou, X. Min, X. Zou, X. Shen, Y. Gong, Y. Zhu, Y. Zhou, Y. Zhong, Y. Hu, Y. Fan, Y. Yu, Y. Yang, Y. Li, Y. Huang, Y. Li, Y. Huang, Y. Xu, Y. Mao, Z. Li, Z. Li, Z. Tao, Z. Ying, Z. Cong, Z. Qin, Z. Fan, Z. Yu, Z. Jiang, and Z. Wu. Minimax-01: Scaling foundation models with lightning attention. arXiv preprint arXiv:2501.08313, 2025. URL https://arxiv.org/abs/2501.08313.

[19] Y. Li, Y. Huang, B. Yang, B. Venkitesh, A. Locatelli, H. Ye, T. Cai, P. Lewis, and D. Chen. Snapkv: Llm knows what you are looking for before generation. arXiv preprint arXiv:2404.14469, 2024.

[20] L. Long, R. Yang, Y. Huang, D. Hui, A. Zhou, and J. Yang. Sliminfer: Accelerating long-context llm inference via dynamic token pruning. In AAAI, 2025.

[21] E. Lu, Z. Jiang, J. Liu, Y. Du, T. Jiang, C. Hong, S. Liu, W. He, E. Yuan, Y. Wang, Z. Huang, H. Yuan, S. Xu, X. Xu, G. Lai, Y. Chen, H. Zheng, J. Yan, J. Su, Y. Wu, N. Y. Zhang, Z. Yang, X. Zhou, M. Zhang, and J. Qiu. Moba: Mixture of block attention for long-context llms. arXiv preprint arXiv:2502.13189, 2025. URL https://arxiv.org/abs/2502.13189.

[22] Qwen Team. Qwen3-Next: Towards ultimate training & inference efficiency. https://qwen.ai/blog?id=e34c4305036ce60d55a0791b170337c2b70ae51d, 2025.

[23] Y. Sun, L. Dong, S. Huang, S. Ma, Y. Xia, J. Xue, J. Wang, and F. Wei. Retentive network: A successor to transformer for large language models. arXiv preprint arXiv:2307.08621, 2023. URL https://arxiv.org/abs/2307.08621.

[24] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, et al. Llama: Open and efficient foundation language models. arXiv preprint arXiv:2302.13971, 2023.

[25] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. arXiv preprint arXiv:2307.09288, 2023.

[26] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. Advances in Neural Information Processing Systems, 30, 2017.

[27] Y. Wang, H. He, S. Bao, H. Wu, H. Wang, Q. Zhu, and W. Che. Proxyattn: Guided sparse attention via representative heads. arXiv preprint arXiv:2509.24745, 2025. URL https://arxiv.org/abs/2509.24745.

[28] B. Xiao, B. Xia, B. Yang, et al. MiMo-V2-Flash technical report. arXiv preprint arXiv:2601.02780, 2026.

[29] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis. Efficient streaming language models with attention sinks. In ICLR, 2024.

[30] G. Xiao, J. Guo, K. Mazaheri, and S. Han. Optimizing mixture of block attention. arXiv preprint arXiv:2511.11571, 2025. URL https://arxiv.org/abs/2511.11571.

[31] R. Xu, G. Xiao, H. Huang, J. Guo, and S. Han. Xattention: Block sparse attention with antidiagonal scoring. In International Conference on Machine Learning, 2025.

[32] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, C. Zheng, D. Liu, F. Zhou, F. Huang, F. Hu, H. Ge, H. Wei, H. Lin, J. Tang, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Zhou, J. Lin, K. Dang, K. Bao, K. Yang, L. Yu, L. Deng, M. Li, M. Xue, M. Li, P. Zhang, P. Wang, Q. Zhu, R. Men, R. Gao, S. Liu, S. Luo, T. Li, T. Tang, W. Yin, X. Ren, X. Wang, X. Zhang, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Zhang, Y. Wan, Y. Liu, Z. Wang, Z. Cui, Z. Zhang, Z. Zhou, and Z. Qiu. Qwen3 technical report. arXiv preprint arXiv:2505.09388, 2025. URL https://arxiv.org/abs/2505.09388.

[33] A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei, H. Lin, J. Yang, J. Tu, J. Zhang, J. Yang, J. Y. Parameter, J. Zhou, J. Lin, K. Dang, K. Lu, K. Bao, K. Yang, L. Yu, M. Li, M. Xue, P. Zhang, Q. Zhu, R. Men, R. Lin, T. Li, T. Tang, T. Xia, X. Ren, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Wan, Y. Liu, Z. Cui, Z. Zhang, and Z. Qiu. Qwen2.5 technical report. arXiv preprint arXiv:2412.15115, 2025. URL https://arxiv.org/abs/2412.15115.

[34] A. Yang, B. Yu, C. Li, D. Liu, F. Huang, H. Huang, J. Jiang, J. Tu, J. Zhang, J. Zhou, J. Lin, K. Dang, K. Yang, L. Yu, M. Li, M. Sun, Q. Zhu, R. Men, T. He, W. Xu, W. Yin, W. Yu, X. Qiu, X. Ren, X. Yang, Y. Li, Z. Xu, and Z. Zhang. Qwen2.5-1m technical report. arXiv preprint arXiv:2501.15383, 2025.

[35] S. Yang, B. Wang, Y. Shen, R. Panda, and Y. Kim. Gated linear attention transformers with hardware-efficient training. In International Conference on Machine Learning, 2024.

[36] S. Yang, B. Wang, Y. Zhang, Y. Shen, and Y. Kim. Parallelizing linear transformers with the delta rule over sequence length. In Advances in Neural Information Processing Systems, 2024.

[37] S. Yang, J. Kautz, and A. Hatamizadeh. Gated delta networks: Improving mamba2 with delta rule. In International Conference on Learning Representations, 2025.

[38] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun. Orca: A distributed serving system for transformer-based generative models. In Operating Systems Design and Implementation, 2022.

[39] J. Yuan, H. Gao, D. Dai, J. Luo, L. Zhao, Z. Zhang, Z. Xie, Y. Wei, L. Wang, Z. Xiao, Y. Wang, C. Ruan, M. Zhang, W. Liang, and W. Zeng. Native sparse attention: Hardware-aligned and natively trainable sparse attention. In Annual Meeting of the Association for Computational Linguistics, 2025.

[40] A. Zeng, B. Xu, B. Wang, C. Zhang, D. Yin, D. Rojas, G. Feng, H. Zhao, H. Lai, H. Yu, H. Wang, J. Sun, J. Zhang, J. Cheng, J. Gui, J. Tang, J. Zhang, J. Li, L. Zhao, L. Wu, L. Zhong, M. Liu, M. Huang, P. Zhang, Q. Zheng, R. Lu, S. Duan, S. Zhang, S. Cao, S. Yang, W. L. Tam, W. Zhao, X. Liu, X. Xia, X. Zhang, X. Gu, X. Lv, X. Liu, X. Liu, X. Yang, X. Song, X. Zhang, Y. An, Y. Xu, Y. Niu, Y. Yang, Y. Li, Y. Bai, Y. Dong, Z. Qi, Z. Wang, Z. Yang, Z. Du, Z. Hou, and Z. Wang. Chatglm: A family of large language models from glm-130b to glm-4 all tools. arXiv preprint arXiv:2406.12793, 2024. URL https://arxiv.org/abs/2406.12793.

[41] Y. Zhang, Z. Lin, X. Yao, J. Hu, F. Meng, C. Liu, X. Men, S. Yang, Z. Li, W. Li, E. Lu, W. Liu, Y. Chen, W. Xu, L. Yu, Y. Wang, Y. Fan, L. Zhong, E. Yuan, D. Zhang, Y. Zhang, Y. T. Liu, H. Wang, S. Fang, W. He, S. Liu, Y. Li, J. Su, J. Qiu, B. Pang, J. Yan, Z. Jiang, W. Huang, B. Yin, J. You, C. Wei, Z. Wang, C. Hong, Y. Chen, G. Chen, Y. Wang, H. Zheng, F. Wang, Y. Liu, M. Dong, Z. Zhang, S. Pan, W. Wu, Y. Wu, L. Guan, J. Tao, G. Fu, X. Xu, Y. Wang, G. Lai, Y. Wu, X. Zhou, Z. Yang, and Y. Du. Kimi linear: An expressive, efficient attention architecture. arXiv preprint arXiv:2510.26692, 2025. URL https://arxiv.org/abs/2510.26692.

[42] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng. SGLang: Efficient execution of structured language model programs. In Advances in Neural Information Processing Systems, 2024.

A. Implementation and deployment details

UniPrefill is implemented and evaluated on top of vLLM v0.16.0 [15], with full support for prefill-decode co-processing and tensor parallelism, making it compatible with standard production deployment configurations. All throughput experiments are conducted under tensor parallelism degree TP = 8, reflecting a typical multi-GPU serving setup in real-world scenarios. The continuous-batching operator is implemented as a set of fused Triton kernels, which are hardware-agnostic by design and theoretically portable across different accelerator platforms. All experiments are conducted under CUDA 12.8.

B. Experiment statistical significance.

Tab. 5 reports results across multiple random seeds, and the consistently stable performance demonstrates that UniPrefill is robust to random seed initialization.

random seed	4K	8K	16K	32K	64K	128K
0	96.53	95.83	95.41	89.77	85.28	79.87
321	96.53	95.83	95.41	89.77	85.28	79.87
3467	96.53	95.83	95.41	89.77	85.28	79.87

Table 5 | Ablation study on different random seed.

C. Limitations and Broader Impacts

This work focuses on accelerating the prefill phase for long-context LLM inference. While UniPrefill delivers consistent speedups across diverse architectures and sequence lengths without observable accuracy degradation, extending the framework to broader inference optimization dimensions—such as decoding acceleration or training-time efficiency—remains an interesting direction for future work. Broader societal considerations, including ethical deployment and safety alignment, are important but lie outside the technical scope of this study.